

Enhancing Collective Communication Efficiency via Multi-Path Ring-Based Algorithms

A Research Proposal for Enhancing Collective Communication
in Modern Data-Center Fabrics

Ibrahim Hamad

Supervised by Dr. Jose Yallouz

Department of Computer Science
University of Haifa

`ibrahim.hamad@cs.haifa.ac.il`

October 2025

Abstract

Collective communication operations, such as All-Reduce, AllGather, and AllToAll, are fundamental building blocks in high-performance computing (HPC) and distributed training frameworks. Their implementation through ring-based algorithms has been widely adopted due to its simplicity, scalability, and efficient bandwidth utilization. Nonetheless, its reliance on a single deterministic path per communication severely impacts resilience and throughput, particularly in modern high-bandwidth networks. Furthermore, the performance of the entire collective operation is determined by the weakest connection in the ring; thereby, the usage of multiple parallel paths for a connection might improve the ring algorithm throughput. This research seeks to demonstrate that multi-path collective communication can achieve superior performance and scalability compared to conventional single-ring implementations, particularly under congested network conditions.

Keywords: Collective Communication, Ring All-Reduce, RDMA, ECMP, Clos/Fat-Tree, Data-Center Networks, Bottleneck Throughput

1 Introduction

Parallel training strategies for LLMs—data (DP), tensor/model (TP), pipeline (PP), and mixture-of-experts (MoE)—induce distinct communication patterns: DP primarily uses All-Reduce for gradients; TP repeatedly invokes All-Reduce/AllGather (and sometimes ReduceScatter); PP is dominated by point-to-point send/receive with occasional collectives; MoE introduces heavy AllToAll for token dispatch.

Collective communication is a fundamental operation in distributed systems and large-scale LLM training. We focus on *All-Reduce*, *ReduceScatter*, *AllGather*, and *AllToAll*; in the large-message regime typical of LLMs these collectives are predominantly bandwidth-bound. The ring algorithm is a widely used implementation for collective operations in practice (e.g., NCCL/Horovod)[1]–[3]. Specifically, the ring All-Reduce algorithm achieves the theoretical minimum amount of data that must be communicated by each node, and is therefore considered throughput-optimal. It forms a logical ring among participating nodes, where each node communicates only with its immediate neighbors, systematically passing and aggregating data around the ring. This design suffers from a latency that increases linearly with the number of nodes. However, for large messages, the algorithm maximizes bandwidth utilization on each network link by ensuring continuous data flow, achieving optimal throughput for large buffer sizes typical in training [1], [2].

The Fat-Tree data center topology is a widely used architecture designed for scalability, fault tolerance, and high bandwidth in large-scale networks. It consists of three hierarchical layers: core, aggregation, and edge switches. Each layer is structured so that every switch has the same number of ports, and multiple pods are formed, where each pod contains both aggregation and edge switches. The Fat-Tree is distinct for its provision of multiple redundant paths between any two servers, enhancing resilience and network capacity. This redundancy ensures that if one path or switch fails, data can still flow through alternative routes, maintaining high availability and reliability in the data center. An important property of the Fat-Tree topology is its high path diversity: for any given source-destination pair, there are various equal-cost paths. This structure facilitates large aggregate bandwidth, non-blocking communication, and significant scalability as data center demands increase [4], [5]. As illustrated in Figure 1, a two-tier Fat-Tree with $k = 8$ offers multiple equal-cost up/down paths that ECMP can exploit.

ECMP provides path diversity in Clos/Fat-Tree fabrics by hashing flows to equal-cost paths; however, a single flow per neighbor exercises only one path. See (2.1.4) for details and implications on ring design.

The goal of this research is to implement a dynamic approach for flow management in ring-based algorithms by adaptively increasing the number of flows used in communication whenever connection bandwidth is underutilized. By detecting when existing flows do not fully saturate the network capacity, the algorithm increases the number of parallel flows between two nodes in the ring, spreading traffic across more equal-cost paths to balance load and alleviate congestion.

In this work, we propose a bandwidth-aware multipath (multi-flow) ring semantics while

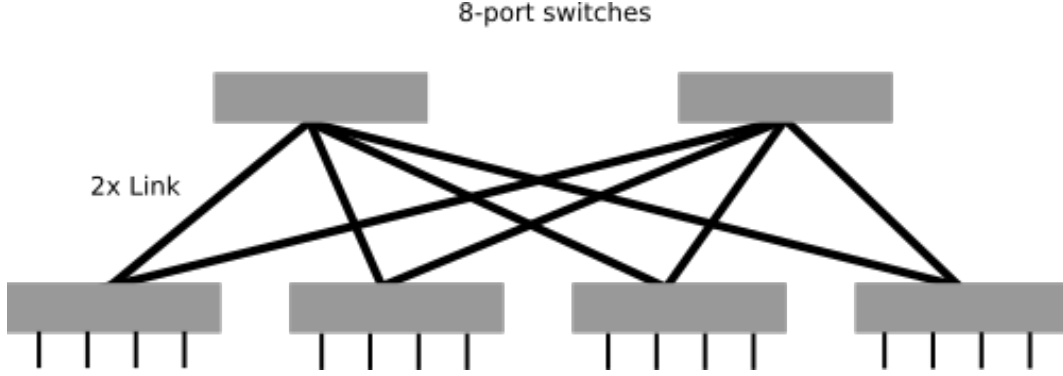


Figure 1: Two-tier Fat-Tree with 8-port switches ($k = 8$; two *switch* levels). Lower tier (edge) switches attach $k/2 = 4$ hosts each and uplink $k/2 = 4$ ports to the upper tier (core). Any host-to-host pair has multiple equal-cost up/down paths (up to $k/2 = 4$ via distinct core switches), enabling ECMP load balancing.

adapting the number of per-neighbor flows to increase the bottleneck *throughput* (achieved bandwidth) on a ring-based collective communication. We aim to establish three main contribution through this study:

- A formal framework to analyze the performance of a ring-based collective communication with the aim of minimizing its completion time.
- The improvement of the bottleneck throughput connection on the ring by the establishment of parallel flows. These flows should be routed through disjoint paths by ECMP mechanism, thus, bypassing the congested links.
- A distributed local mechanism to adjust the transmission throughput of each node in the ring solely by sampling the incoming traffic throughput in the node. This distributed mechanism based on the local information of the nodes aims to maximize the total ring throughput and reduce the completion time of the collective operation.

2 Background

2.1 All-Reduce over Rings

This section recalls the basic ring-based implementation of the *All-Reduce* collective used for gradient synchronization in data-parallel training, and sets the notation used to analyze its cost.

2.1.1 Model and Notation

We consider $P \geq 2$ processing units (ranks) indexed $\{p_0, p_1, \dots, p_{P-1}\}$. Each rank p_i connects to its neighbors $p_{(i+1) \bmod P}$ and $p_{(i-1) \bmod P}$, forming a directed logical ring. Each rank holds a buffer of size M bytes that must be reduced across ranks and then replicated to all ranks. Communication proceeds in fixed-size *chunks* of M/P bytes per step via neighbor-to-neighbor

transfers only. We use the standard α - β cost model: each step incurs startup latency α and transmission time proportional to size with factor β (seconds per byte) [1]. For later use, we write $\bar{B}_e$ for the steady-state *application-layer throughput* on logical-ring edge e (bytes/s); a more precise definition is given in Section 3.

2.1.2 Ring All-Reduce (Reduce-Scatter + All-Gather)

Figure 2 illustrates the two-phase pipeline for $P = 4$: Fig. (a) shows a time-stepped three-worker Ring All-Reduce schedule adapted from [6], and Fig. (b) shows the logical partitioning and flow of chunks. The classic Ring All-Reduce algorithm has been widely discussed in the literature [1], [2], [7].

Ring All-Reduce consists of two pipeline phases:

1. **Reduce-Scatter:** across $(P - 1)$ steps, each rank sends one chunk to its clockwise neighbor and receives one from its counter-clockwise neighbor, accumulating a distinct chunk index per step.
2. **All-Gather:** in the following $(P - 1)$ steps, the reduced chunks circulate so that all ranks reconstruct the full reduced buffer.

During the Reduce-Scatter phase, each rank locally reduces the received chunk with its own data and forwards the updated partial result; during the All-Gather phase, no further reductions are performed and chunks are simply forwarded until every rank holds the complete reduced buffer, as in standard Ring All-Reduce descriptions [1], [2], [7]. Thus a ring of P ranks performs $2(P - 1)$ communication steps in total. Each rank sends and receives $\frac{2(P-1)}{P}M$ bytes in total (approaching $2M$ for large P); for sufficiently large M the pipeline sustains continuous link utilization on each link [1], [2], [7].

2.1.3 Communication Cost and Completion Time (Bottleneck Model)

Let $B_\star \triangleq \min_e \bar{B}_e$ be the per-step bottleneck across logical-ring edges. Under the α - β model and full pipelining over chunks of size M/P [1], [2], the completion time is

$$T_{\text{ring}}(M, P) = 2(P - 1)\alpha + \frac{2(P - 1)}{P} \cdot \frac{M}{B_\star}. \quad (1)$$

For small M the latency term $2(P - 1)\alpha$ dominates; for large M throughput dominates via $B_\star$. A single slow edge (small $B_\star$) throttles the entire pipeline. This expression follows the standard cost analysis of ring-based collectives in the MPI and deep learning literature [1], [2], [7]. We later extend it to a per-neighbor multi-flow operation, yielding $T_{\text{ring}}(M, P, \mathbf{k})$ with bottleneck $B_\star(\mathbf{k})$ (Eq. 2).

2.1.4 Implications on Multipath Fabrics

In Fat-Tree networks, ECMP maps each transport *flow* (5-tuple) to a single path of equal-cost [4], [8]. A classic ring maintains one flow per neighbor, so only one path is used per hop; therefore $B_\star$ can be dictated by an unfavorable selection of congested paths. To exploit path

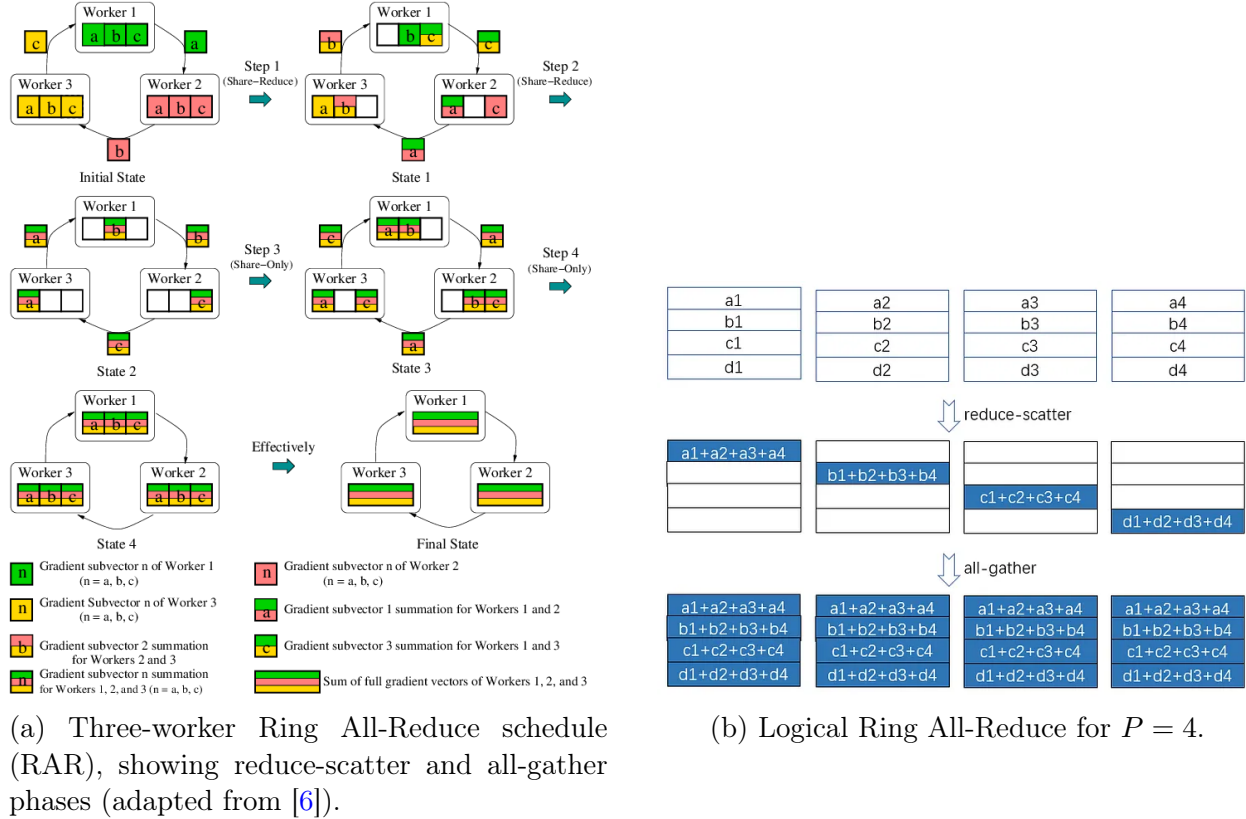


Figure 2: Ring All-Reduce execution. Panel (a) shows a time-stepped three-worker schedule that makes the reduce-scatter and all-gather phases explicit, while panel (b) shows the logical partitioning and flow of chunks for $P = 4$.

diversity while preserving the semantics of the ring, we allow each logical-ring edge e to use k_e parallel flows (distinct 5-tuples), yielding a bottleneck $B_\star(\mathbf{k})$. For stability, $\{k_e\}$ are adjusted only at coarse-grained control intervals (e.g., between groups of collective iterations).

3 Problem Formulation

In a ring-based collective, every message chunk must traverse all neighbor-to-neighbor connections along the ring. As a result, the overall throughput of the collective is bottlenecked by the slowest neighbor link. Modern Clos/Fat-Tree data-center fabrics expose multiple equal-cost physical paths between any two ranks via ECMP, but a conventional ring typically uses a single transport flow per neighbor, so its throughput may be limited by congestion on a single unfavorable path. To reason about using multiple flows per neighbor while preserving ring semantics, we formalize the underlying fabric and the induced logical neighbor links as follows.

We model the fabric as a directed multigraph $G = (V, E)$ where each link $e \in E$ is associated with a capacity c_e (bytes/s). For a pair of nodes $u, v \in V$, let $\text{Paths}_{u,v}(G)$ be the set of all simple directed paths $P_{u,v}$ in G between u and v . Note that there might be several paths in

$\text{Paths}_{u,v}(G)$ and at least one in case G is connected. Assume bandwidth $B(P_{u,v})$ as the maximum feasible bandwidth along path $P_{u,v} \in \text{Paths}_{u,v}(G)$. We define logical connection $L_{u,v}$ between a pair of nodes $u, v \in V$ as a subset of $\text{Paths}_{u,v}(G)$, i.e. $L_{u,v} \subseteq \text{Paths}_{u,v}(G)$. A logical ring $R(v_0, \dots, v_k)$ over nodes $(v_0, \dots, v_k)$ induces a set of logical connections between two cyclic consecutive nodes in $(v_0, \dots, v_k)$, i.e. $\{L_{v_0,v_1}, \dots, L_{v_k,v_0}\}$.

Definition 1 (Achieved bandwidth of a logical connection). *Given a logical connection $L_{u,v}$ and the set of bandwidth $B(P_{u,v})$ (bytes/s) of $B(P_{u,v})$. The achieved bandwidth on $L_{u,v}$ is the sum of the set of bandwidth $B(P_{u,v})$*

$$\bar{B}_{L_{u,v}} \triangleq \sum_{P_{u,v} \in L_{u,v}} B(P_{u,v}).$$

Definition 2 (Ring bandwidth). *Given a logical ring $R(v_0, \dots, v_k)$, its bandwidth is defined by the bottleneck bandwidth among its logical connections.*

$$\bar{B}_{R(v_0, \dots, v_k)} \triangleq \min_{L_{u,v} \in R(v_0, \dots, v_k)} \bar{B}_{L_{u,v}}.$$

Definition 3 (Per-node input and output rates). *For each node v , $\bar{B}_{L_{u,v}}$ and $\bar{B}_{L_{v,w}}$ denote its incoming and outgoing logical connection achieved bandwidth along the ring, respectively, and define*

$$\lambda_v^{\text{in}} \triangleq \bar{B}_{L_{u,v}}, \quad \lambda_v^{\text{out}} \triangleq \bar{B}_{L_{v,w}},$$

as the corresponding input and output rates (bytes/s).

These per-node rates will later serve as local measurements for the distributed controller that adjusts the logical connection size $|L_{u,v}|$ using only information observed at each node. In particular, situations where $\lambda_v^{\text{in}} > \lambda_v^{\text{out}}$ indicate that the outgoing neighbor link of node v is locally limiting the throughput, motivating an increase in the number of parallel flows on that link in our design.

Completion time objective. For message size M (bytes) and a logical ring $R(v_0, \dots, v_P)$, the pipelined ring completion time is

$$T_{R(v_0, \dots, v_P)}(M) = 2(P-1)\alpha + \frac{2(P-1)}{P} \cdot \frac{M}{\bar{B}_{R(v_0, \dots, v_P)}}, \quad (2)$$

where α is the per-step startup latency in the α - β model (seconds) according to [1], [2].

In this proposal we aim to optimize the completion time $T_{R(v_0, \dots, v_P)}(M)$ by designing an efficient distributed controller that adjusts the ring bandwidth $\bar{B}_{R(v_0, \dots, v_k)}$ based only on the local rates $(\lambda_v^{\text{in}}, \lambda_v^{\text{out}})$ of each node v .

3.1 RDMA connection

In many HPC and distributed-training deployments, the logical ring edges are carried over Remote Direct Memory Access (RDMA) transports (InfiniBand or RoCEv2) to reduce CPU

cycles via kernel-bypass and zero-copy and to increase achieved bandwidth; alternatives such as TCP/IP stacks are also used in practice. In this work we will implement the ring over RDMA for these efficiency reasons, while the cost decomposition in Eq. (1) remains unchanged (the per-edge achieved bandwidth still determines the bottleneck $B_\star$) [3], [9].

3.1.1 RDMA Basics and Purpose

RDMA enables a process to access a remote process’s registered memory directly via the NIC. We use Reliable Connection (RC) Queue Pairs (QPs), which provide reliable, in-order delivery *per QP*. Each rank registers contiguous device buffers (obtaining LKey/RKey) and exchanges virtual addresses and RKeys with its two neighbors. Two communication types are relevant [9]:

- **Two-sided *SEND/RECV***: both endpoints post matching work requests; simple flow control, CPU involvement at both endpoints.
- **One-sided *WRITE/READ***: the initiator NIC directly writes to or reads from the peer’s registered memory; the remote CPU is off the data path. *WRITE with immediate* additionally posts a lightweight signal to the receiver’s completion queue.

3.1.2 Implementing the Ring with RDMA READ/WRITE

Let ranks be indexed modulo P , with $\text{next}(p) = (p+1) \bmod P$ and $\text{prev}(p) = (p-1) \bmod P$. The message of size M is partitioned into P chunks of size M/P and pipelined across steps.

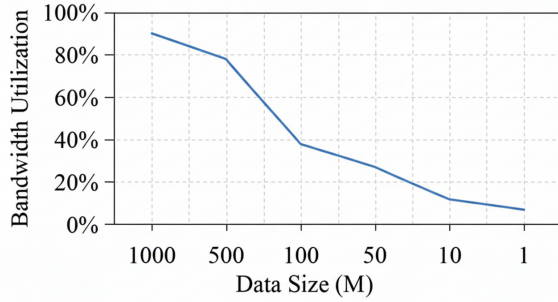
WRITE–push (producer-driven). At step s , rank p issues an *RDMA_WRITE* of chunk j into a pre-allocated window at $\text{next}(p)$ (optionally with immediate to signal arrival); the receiver reduces the arrived chunk into its local accumulator for index j .

READ–pull (consumer-driven). At step s , rank p issues an *RDMA_READ* to fetch chunk j from $\text{prev}(p)$ and performs the local reduction; reads require no pre-posted receives; pacing is entirely local: the consumer keeps a small window w of in-flight RDMA READs and issues a new READ only when a completion returns (i.e., a credit is freed).

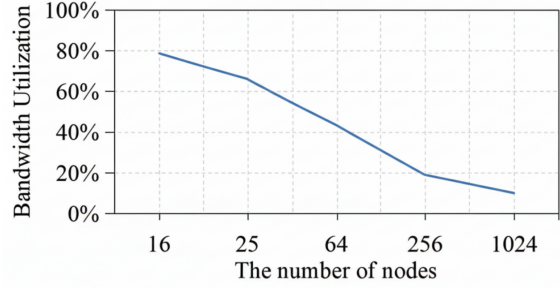
RC guarantees in-order delivery within a QP. To avoid cross-QP reordering, each chunk is issued as a single work-request chain on one QP, with per-chunk sequence numbers checked on completion.

3.1.3 Multipath Implementation

On RoCEv2 Clos/Fat-Tree fabrics, the RC QP number constitutes an input to the ECMP hash function causes ECMP to spread these different QP flows over multiple equal-cost physical paths [8], [10]. In our setting, each logical ring edge between two neighboring ranks may use several parallel QPs (several flows). The total bandwidth of that edge is the bandwidth obtained by all its QPs together. Intuitively, using more QPs increases the chance that ECMP will route some of the flows through less congested physical paths, which can improve the effective throughput of that edge.



(a) Utilization vs. data size (adapted from IBING).



(b) Utilization vs. number of nodes (adapted from IBING).

Figure 3: Single-flow ring underutilization in practice: degradation with smaller M and larger P , aligning with the $B_\star$ bottleneck model.

The ring algorithm itself remains simple: each data flow is pinned to a single QP, so RC ordering guarantees in-order delivery per flow. Thus, our algorithm should divide the chunk into proper sub-chunks associated to a specific flow. Accordingly, the receiver should track only the flow order. ECMP runs transparently in the network and decides which physical path each QP uses; our algorithm only controls how many QPs are opened between neighbors and how they are used.

4 Preliminary work

Modern training clusters often operate under oversubscription and transient failures. In such settings, the classic All-Reduce ring can suffer from cascading dependencies (“chain blocking”): congestion or a slow/failing hop throttles the entire pipeline. A recent work [11] shows that All-Reduce completion time and throughput can be detrimentally impacted by contention and failure scenarios.

A contributing root cause is how equal-cost multipath (ECMP) forwards traffic: a single transport flow is hashed onto one equal-cost path. Thus, the common single-flow-per-neighbor ring employs only one path per hop; unfavorable placements on congested links degrade the per-step bottleneck $B_\star$ that dictates completion time. Exploiting path diversity requires multiple parallel flows so that ECMP can distribute load across available paths [8], [12].

This observation motivates our design: preserve ring semantics but open k parallel RDMA flows per neighbor and adapt k online using a simple throughput-based rule evaluated over coarse-grained measurement windows. By raising the per-edge achieved bandwidth $\bar{B}_e^{(k_e)}$ and, in turn, the bottleneck $B_\star(\mathbf{k})$, we aim to reduce $T_{\text{ring}}(M, P, \mathbf{k})$ on Clos/Fat-Tree fabrics with ECMP and ECN-based congestion control [8], [10].

Empirically (adapted from IBING [7]), single-flow rings exhibit (i) utilization drop as message size decreases (latency/overheads dominate), and (ii) further degradation as the number of

ranks P grows (the bottleneck $B_\star$ dominates)—both consistent with Eq. 1 and motivating per-neighbor multi-flow (see Fig. 3).

Preserving ring semantics while opening k per-neighbor RDMA flows (distinct 5-tuples) should improve $B_e^{(k_e)}$ and thus $B_\star(\mathbf{k})$, reducing $T_{\text{ring}}(M, P, \mathbf{k})$ on Clos/Fat-Tree fabrics [10].

5 Related Work

Ring-based collectives are widely used in MPI and deep learning frameworks; classic analyses and optimizations appear in [1] and production practice in [2], [3]. Under congestion and failures, ring All-Reduce can suffer from “chain blocking” and heavy-tailed completion times; alternative designs such as the Resilient Allreduce Tree (RAT) improve robustness and tail behavior under such conditions [11].

Beyond single-job collectives, Yu et al. [6] study how to schedule multiple Ring All-Reduce learning jobs in multi-tenant GPU clusters, focusing on bandwidth allocation and job completion time rather than modifying the per-ring communication pattern itself.

On multipath fabrics, ECMP maps each flow (5-tuple) to one equal-cost path [8]. RDMA deployments in Clos/Fat-Tree data centers typically rely on ECN-based congestion control for stability [10]. At the transport layer, systems such as Virtuoso propose practical, scalable multi-path RDMA mechanisms that stripe traffic across several paths while remaining compatible with existing RDMA stacks [12]. In contrast, our work operates at the collective-algorithm layer: we assume a standard RoCEv2+ECMP+ECN fabric and focus on how a ring All-Reduce can open and adapt the number of RDMA flows per logical ring edge. The goal is to raise the application-level bottleneck throughput $B_\star(\mathbf{k})$ of the ring, rather than redesign the underlying RDMA transport.

Topologically, Clos/Fat-Tree networks provide rich path diversity and scale-out with commodity switching [4], [5]. Alternative high-scale designs (e.g., DCell, BCube, Dragonfly) illustrate other path-diverse fabrics [13]–[15]; we focus on Clos/Fat-Tree with ECN/DCQCN, where ring semantics and ECMP interplay directly.

A related strand of work optimizes ring-based collectives by modifying the logical topology, for example by constructing multiple edge-disjoint or ingress-balanced rings to alleviate congestion hot spots in Clos/Fat-Tree fabrics [7]. Other schemes replace rings altogether with more resilient tree-based structures such as RAT [11]. These approaches change the collective topology, whereas we keep a single logical ring and instead multiplex several RDMA flows per logical edge to exploit ECMP path diversity.

A complementary line of work, known as *collective synthesis*, designs collective algorithms automatically. Given a network topology and a target operation such as All-Reduce, systems like SCCL and TACOS [16], [17] search for communication schedules that nearly saturate the available bandwidth. Our work is orthogonal: we keep the ring algorithm fixed and instead exploit path diversity on each logical ring edge via adaptive multi-flow RDMA, so that a multi-path ring can serve as a building block within synthesized schedules.

6 Research Plan

The proposed research will be carried out in four stages, combining an instrumented single-flow baseline, a multi-flow ring with a local controller, and an experimental evaluation on Clos/Fat-Tree fabrics:

1. **Stage 1 – Single-flow baseline.** Implement a single-flow ring All-Reduce over RDMA (Section 3.1) and instrument it to measure per-edge throughput $\bar{B}_e$, bottleneck throughput $B_\star$, and completion time $T_{\text{ring}}(M, P)$ for different message sizes M and numbers of ranks P . Use these measurements to validate and calibrate the model and to select practical defaults for chunk size and the number of in-flight operations.
2. **Stage 2 – Multi-flow ring and local controller.** Extend the baseline to support $k_e \geq 1$ parallel RDMA flows per logical ring edge $e \in E_R$, using distinct 5-tuples (RC QPs) so that ECMP can exploit multiple physical paths. Define a simple chunk-to-QP assignment that preserves per-chunk ordering, and implement a local throughput-based controller at each rank that periodically estimates $(\lambda_p^{\text{in}}, \lambda_p^{\text{out}})$ and adjusts k_e only at coarse-grained control intervals.
3. **Stage 3 – Experimental evaluation on multipath fabrics.** Evaluate the single-flow and multi-flow rings on a Clos/Fat-Tree fabric with ECMP and ECN-based congestion control. Measure $\bar{B}_e^{(k_e)}$, $B_\star(\mathbf{k})$, and $T_{\text{ring}}(M, P, \mathbf{k})$ for different $(M, P, \mathbf{k})$ both without cross-traffic and under controlled background load. Compare median and tail completion times, bandwidth utilization, and the overhead of maintaining multiple flows (QPs, CPU/NIC usage).
4. **Stage 4 – Analysis, guidelines, and extensions.** Analyze the experimental results and derive design guidelines: for which ranges of (M, P) and network conditions a multi-flow ring is beneficial, and how many flows per neighbor are typically sufficient. If time permits, extend the approach to other bandwidth-bound collectives (e.g., AllGather) and evaluate robustness under transient link slowdowns or failures that emulate realistic data-center behavior.

Acknowledgments

References

- [1] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005. DOI: [10.1177/1094342005051521](https://doi.org/10.1177/1094342005051521).
- [2] A. Sergeev and M. D. Balso, *Horovod: Fast and easy distributed deep learning in TensorFlow*, 2018. arXiv: [1802.05799 \[cs.LG\]](https://arxiv.org/abs/1802.05799). [Online]. Available: <https://arxiv.org/abs/1802.05799>.
- [3] NVIDIA Corporation. “NCCL documentation: Environment variables.” Describes use of InfiniBand/RoCE when available and TCP fallback for collectives including ring. (2021), [Online]. Available: <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html> (visited on 11/08/2025).
- [4] C. E. Leiserson, “Fat-trees: Universal networks for hardware-efficient supercomputing,” *IEEE Transactions on Computers*, vol. 34, no. 10, pp. 892–901, 1985. DOI: [10.1109/TC.1985.6312192](https://doi.org/10.1109/TC.1985.6312192).
- [5] M. Al-Fares, A. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” in *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*, 2008, pp. 63–74. DOI: [10.1145/1402958.1402967](https://doi.org/10.1145/1402958.1402967).
- [6] M. Yu, B. Ji, H. Rajan, and J. Liu, “On scheduling ring-all-reduce learning jobs in multi-tenant GPU clusters with communication contention,” in *Proceedings of the 23rd International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing (MobiHoc ’22)*, Seoul, Republic of Korea: Association for Computing Machinery, 2022, pp. 21–30.
- [7] R. Zong, J. Zhang, Z. Tang, and K. Li, “IBing: An efficient interleaved bidirectional ring all-reduce algorithm for gradient synchronization,” *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 1, 35:1–35:23, 2025. DOI: [10.1145/3711818](https://doi.org/10.1145/3711818).
- [8] C. Hopps, “Analysis of an equal-cost multi-path algorithm (ECMP),” RFC Editor, RFC 2992, Nov. 2000. DOI: [10.17487/RFC2992](https://doi.org/10.17487/RFC2992). [Online]. Available: <https://www.rfc-editor.org/info/rfc2992>.
- [9] *Infiniband architecture specification, volume 1*, version Release 1.3, Defines RC QP semantics, memory registration, and one-/two-sided verbs, InfiniBand Trade Association, 2015. [Online]. Available: <https://www.infinibandta.org/> (visited on 11/08/2025).
- [10] Y. Zhu, H. Eran, D. Firestone, *et al.*, “Congestion control for large-scale RDMA deployments,” in *Proceedings of the ACM SIGCOMM 2015 Conference*, 2015, pp. 523–536. DOI: [10.1145/2785956.2787484](https://doi.org/10.1145/2785956.2787484).
- [11] X. Wan, H. Zhang, H. Wang, S. Hu, J. Zhang, and K. Chen, “RAT: Resilient allreduce tree for distributed machine learning,” in *Proceedings of the 4th Asia-Pacific Workshop on Networking (APNet ’20)*, Demonstrates chain blocking and throughput degradation of ring under oversubscription, ACM, 2020. DOI: [10.1145/3411029.3411037](https://doi.org/10.1145/3411029.3411037). [Online]. Available: <https://dl.acm.org/doi/10.1145/3411029.3411037>.
- [12] R. Luo, M. Kermani, P. Mishra, A. Krishnamurthy, J. R. Lorch, W. Bai, H. Tang, P. Cheng, *et al.*, *Virtuoso: Practical and scalable multipath RDMA in the public cloud*,

- Shows production multipath RDMA design; relevant to parallel per-neighbor flows, 2020. DOI: [10.48550/arXiv.2009.00243](https://doi.org/10.48550/arXiv.2009.00243). arXiv: [2009.00243](https://arxiv.org/abs/2009.00243) [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2009.00243>.
- [13] C. Guo, H. Wu, K. Tan, L. Shi, Y. Zhang, and S. Lu, “DCell: A scalable and fault-tolerant network structure for data centers,” in *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*, 2008, pp. 75–86. DOI: [10.1145/1402958.1402968](https://doi.org/10.1145/1402958.1402968).
 - [14] C. Guo, G. Lu, D. Li, *et al.*, “BCube: A high performance, server-centric network architecture for modular data centers,” in *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, 2009, pp. 63–74. DOI: [10.1145/1594977.1592577](https://doi.org/10.1145/1594977.1592577).
 - [15] J. Kim, W. J. Dally, S. Scott, and D. Abts, “Technology-driven, highly-scalable dragonfly topology,” in *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, 2008, pp. 77–88. DOI: [10.1109/ISCA.2008.19](https://doi.org/10.1109/ISCA.2008.19).
 - [16] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, “Synthesizing optimal collective algorithms,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 2021)*, ACM, 2021.
 - [17] W. Won, M. Elavazhagan, S. Srinivasan, S. Gupta, and T. Krishna, “TACOS: Topology-aware collective algorithm synthesizer for distributed machine learning,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2024)*, IEEE, 2024, pp. 856–870.